In [240…

```
#import libraries
!pip install numpy
!pip install pandas
!pip install matplotlib
!pip install seaborn
!pip install sqlite3
```

```
Defaulting to user installation because normal site-packages is not writeable
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
Requirement already satisfied: numpy in c:\users\hp\appdata\roaming\python\python310\site-packages (2.2.6)
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pandas in c:\users\hp\appdata\roaming\python\python310\site-packages (2.3.3)
Requirement already satisfied: numpy>=1.22.4 in c:\users\hp\appdata\roaming\python\python310\site-packages (from pandas) (2.2.
6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\hp\appdata\roaming\python\python310\site-packages (from panda
s) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\appdata\roaming\python\python310\site-packages (from pandas) (2026.
2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp\appdata\roaming\python\python310\site-packages (from pandas) (202
6.2)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\roaming\python\python310\site-packages (from python-dateutil>=2.
8.2->pandas) (1.17.0)
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: matplotlib in c:\users\hp\appdata\roaming\python\python310\site-packages (3.10.9)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib)
(1.3.2)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib) (0.
12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotli
b) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotli
b) (1.5.0)
Requirement already satisfied: numpy>=1.23 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib) (2.
2.6)
Requirement already satisfied: packaging>=20.0 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib)
(26.2)
Requirement already satisfied: pillow>=8 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib) (12.3.
0)
Requirement already satisfied: pyparsing>=3 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib) (3.
3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplot
lib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\roaming\python\python310\site-packages (from python-dateutil>=2.
7->matplotlib) (1.17.0)
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: seaborn in c:\users\hp\appdata\roaming\python\python310\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\hp\appdata\roaming\python\python310\site-packages (from seabor
n) (2.2.6)
Requirement already satisfied: pandas>=1.2 in c:\users\hp\appdata\roaming\python\python310\site-packages (from seaborn) (2.3.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\hp\appdata\roaming\python\python310\site-packages (from seab
orn) (3.10.9)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib!
=3.6.1,>=3.4->seaborn) (1.3.2)
Requirement already satisfied: cycler>=0.10 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib!=3.
6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotli
b!=3.6.1,>=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotli
b!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib!=
3.6.1,>=3.4->seaborn) (26.2)
Requirement already satisfied: pillow>=8 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib!=3.6.1,
>=3.4->seaborn) (12.3.0)
Requirement already satisfied: pyparsing>=3 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplotlib!=3.
6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\hp\appdata\roaming\python\python310\site-packages (from matplot
lib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\hp\appdata\roaming\python\python310\site-packages (from pandas>=1.2->se
aborn) (2026.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\hp\appdata\roaming\python\python310\site-packages (from pandas>=1.2->
seaborn) (2026.2)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\roaming\python\python310\site-packages (from python-dateutil>=2.
7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
```

```
ERROR: Could not find a version that satisfies the requirement sqlite3 (from versions: none)

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
ERROR: No matching distribution found for sqlite3
```

In [61]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

In [ ]:
```python
#1. import database/ data
```

In [66]:
```python
conn = sqlite3.connect('customer_churn.db')

sql_query = """
            select name
            from sqlite_master
            where type = 'table';
"""

tables = pd.read_sql(sql_query, conn) #here it will gather the data from the source that we have mentioned and the connection

#creation of dataframe for each table.
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

In [67]:
```python
#this is just for our knowledge:


#we havent seen the tables in the database and we want to see the columns in the datatables hence we can see this query.
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    #get the column information
    columns_query = f"PRAGMA table_info({table_name});" #pragma is a special command tpo bring in the table info to the answer
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

#close connection
conn.close()
```

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type', 'contract_type', 'cancellation_dat
e', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

In [68]:
```python
df_db_customer.head() #.head is a general practice so that whenever we bring in any table we  have its intial rows to understa
```

Out[68]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	None	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	None	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [69]:
```python
#2. DATA CLEANING:->
df_db_customer.info() #this will help us to get the overview of the values that are missing.
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   customerid  21 non-null     object
 1   name        21 non-null     object
 2   country     18 non-null     object
 3   state       21 non-null     object
 4   gender      21 non-null     object
 5   dob         21 non-null     object
 6   interests   4 non-null      object
 7   pincode     0 non-null      object
dtypes: object(8)
memory usage: 1.4+ KB
```

In [70]:
```
# here we have observed that, we have to drop column interest and pincode and rename the column from name to customer name.
#2. Fill the null value that we have observed total enteries were 21 but in some cases there are only 17-18 values.
#3. changing the dob to date datatype.
#4. fixing the gender column basically doning the DATA STANDARDISATION.
```

In [71]:
```
df_db_customer.head()
```

Out[71]:

	customerid	name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	None	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	None	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [72]:
```
df_db_customer.tail()
#here we observed that the gender was male and female but at many places there were cases where it was written men and women.
```

Out[72]:

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	None	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	None	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	None	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	None	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	None	None

In [73]:
```
#RENAMING THE COLUMN
df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace = True)
#the chngese using the rename parameter are not permanent hence we used inplace which is by default false to fix the value in
```

In [74]:
```
df_db_customer.head()
```

Out[74]:

	customerid	customer_name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	None	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	None	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [75]:
```
#DROPPING THE COLUMNS - interest and pincode.
#df_db_customer.drop(df_db_customer.columns[-2:],axis = 1)
#can be written in the easier way as well
df_db_customer.drop(columns=['interests' , 'pincode'], inplace = True)
#this is way of deleting with the column mentioning.
```

In [76]:
```
df_db_customer.head()
```

Out[76]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00

In [77]:
```
#changing the datatype from object to the date- dob
#another way of changing the datatype without the datatype:
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
#here the inplace method is not recommended because this will return a whole new series instead of modifying the values.
```

In [78]:
```
df_db_customer.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype         
---  ------         --------------  -----         
 0   customerid     21 non-null     object        
 1   customer_name  21 non-null     object        
 2   country        18 non-null     object        
 3   state          21 non-null     object        
 4   gender         21 non-null     object        
 5   dob            21 non-null     datetime64[ns]
dtypes: datetime64[ns](1), object(5)
memory usage: 1.1+ KB
```

In [ ]:
```
df_db_customer.unique() #this will give us the unique values in the column and the values that need to be changed.
```

In [80]:
```
#DATA STANDARDIZATION: changing the values of men and women to male and female:
#we will use the .replace method
df_db_customer['gender'].replace({'Men': 'Male', 'Women': 'Female'},inplace=True) #the replacing value go in the key value pai
```

```
C:\Users\hp\AppData\Local\Temp\ipykernel_10316\725134017.py:3: FutureWarning: A value is trying to be set on a copy of a DataFr
ame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are set
ting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = d
f[col].method(value) instead, to perform the operation inplace on the original object.


  df_db_customer['gender'].replace({'Men': 'Male', 'Women': 'Female'},inplace=True) #the replacing value go in the key value pa
ir.
```

In [81]:
```
# UPDATING THE MISSING VALUES
#WE CAN FIND THE MISSING VALUES:
#1. USING the info method
#2. using the .na method.

df_db_customer['country'].isna() #True means null.
```

Out[81]:
```
0     False
1     False
2     False
3     False
4     False
5      True
6     False
7     False
8      True
9     False
10    False
11    False
12     True
13    False
14    False
15    False
16    False
17    False
18    False
19    False
20    False
Name: country, dtype: bool
```

In [42]:
```
#fixing the missing values: for the country
df_db_customer[df_db_customer['country'].isna()]
```

Out[42]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th> <th>8</th> <th>12</th> </tbody> <tbody></tbody>

0013-MHZWF	durga	None		Delhi	Women	1988-12-10
0015-UOCOJ	maya	None		Kathmandu	Women	1985-07-07
0018-NYROU	chitra	None		Telangana	Female	2004-12-01

In [46]:
```
#state and country - unique value pair
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('state')['country'].to_dict()
#this first drops the value that are not associated to each other and after seeing the further values it created a realtion an


df_db_customer['country'] = df_db_customer['country'].fillna(
    df_db_customer['state'].map(state_country_mapping)
)
```

In [47]:
```
# so after the transformation there should be no null values here: top verify it we will:
df_db_customer[df_db_customer['country'].isna()]
```

Out[47]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> </tbody> <tbody></tbody>

In [48]:
```
df_db_customer[['country','state']]
```

Out[48]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>country</th> <th>state</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

India	Maharashtra
India	Karnataka
India	Delhi
India	Nagaland
India	Delhi
India	Delhi
India	Meghalaya
India	Rajasthan
Nepal	Kathmandu
Nepal	Kathmandu
India	Maharashtra
India	Karnataka
India	Telangana
India	Meghalaya
India	Uttar Pradesh
India	Delhi
India	Meghalaya
India	Rajasthan
India	Telangana
India	Uttar Pradesh
India	Maharashtra

In [49]:
```
# TABLE 2: SUBSCRIPTION TABLE->
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     object
 1   subscription_start_date  21 non-null     object
 2   subscription_type        21 non-null     object
 3   renewal_date             21 non-null     object
 4   plan_type                21 non-null     object
 5   contract_type            21 non-null     object
 6   cancellation_date        6 non-null      object
 7   cancellation_reason      6 non-null      object
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

In [83]:
```
df_db_subscription.head()
```

Out[83]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	None	None	13.99	627	12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	None	None	6.99	210	34
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	None	None	22.99	1725	8
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

In [84]:
```
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     object
 1   subscription_start_date  21 non-null     object
 2   subscription_type        21 non-null     object
 3   renewal_date             21 non-null     object
 4   plan_type                21 non-null     object
 5   contract_type            21 non-null     object
 6   cancellation_date        6 non-null      object
 7   cancellation_reason      6 non-null      object
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: float64(1), int64(2), object(8)
memory usage: 1.9+ KB
```

In [86]:
```
date_col = ['subscription_start_date','renewal_date','cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     object
 1   subscription_start_date  21 non-null     datetime64[ns]
 2   subscription_type        21 non-null     object
 3   renewal_date             21 non-null     datetime64[ns]
 4   plan_type                21 non-null     object
 5   contract_type            21 non-null     object
 6   cancellation_date        6 non-null      datetime64[ns]
 7   cancellation_reason      6 non-null      object
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[ns](3), float64(1), int64(2), object(5)
memory usage: 1.9+ KB
```

In [87]:
```
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   customerid      9 non-null      object
 1   complaint_date  9 non-null      object
 2   escalations     9 non-null      object
 3   csat_score      9 non-null      int64 
 4   col_1           0 non-null      object
 5   comment         4 non-null      object
dtypes: int64(1), object(5)
memory usage: 560.0+ bytes
```

In [90]:
```
df_db_support.drop(columns = ['col_1','comment'], inplace = True)
```

In [92]:
```
df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   customerid      9 non-null      object
 1   complaint_date  9 non-null      object
 2   escalations     9 non-null      object
 3   csat_score      9 non-null      int64 
dtypes: int64(1), object(3)
memory usage: 416.0+ bytes
```

In [93]:
```
df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date'])

df_db_support.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype         
---  ------          --------------  -----         
 0   customerid      9 non-null      object        
 1   complaint_date  9 non-null      datetime64[ns]
 2   escalations     9 non-null      object        
 3   csat_score      9 non-null      int64         
dtypes: datetime64[ns](1), int64(1), object(2)
memory usage: 416.0+ bytes
```

In [94]:
```
#3. FEATURE ENGINEERING AND DATA ANALYSIS:
```

In [95]:
```
#here we will use the churn_flag and if the value is present then the value will be 1 else 0.

df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date'].notna(),1,0)
```

In [96]:
```
#creating a new column using the existing column.
df_db_subscription.head()
```

Out[96]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	churn_score	churn_flag
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	None	13.99	627	12	0
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	None	6.99	210	34	0
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	None	22.99	1725	8	0
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

In [98]:
```
#here we have join operation in pandas as well.
df_db_subscription.merge(df_db_customer, on = 'customerid', how = 'left')
```

Out[98]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	None	13.99	627	12	0	keshav	India	Maharashtra	Male	1982-04-12
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1	raghav	India	Karnataka	Male	1995-11-23
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	None	6.99	210	34	0	lalita	India	Delhi	Female	1978-02-15
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	None	22.99	1725	8	0	mohan	India	Nagaland	Male	2001-08-30
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1	mira	India	Delhi	Female	1990-05-05
0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	Annual	NaT	None	17.99	720	22	0	durga	None	Delhi	Female	1988-12-10
0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	Monthly	2024-11-15	Not enough content	8.99	230	79	1	mina	India	Meghalaya	Female	1976-09-21
0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	Annual	NaT	None	22.99	1840	5	0	madan	India	Rajasthan	Male	1999-03-14
0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	Monthly	NaT	None	13.99	240	34	0	maya	None	Kathmandu	Female	1985-07-07
0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	Annual	NaT	None	6.99	335	41	0	arjun	Nepal	Kathmandu	Male	1993-10-29
0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	Annual	NaT	None	22.99	1610	14	0	shiva	India	Maharashtra	Male	1997-01-22
0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	Monthly	2024-05-01	Poor streaming quality	7.99	270	83	1	rangadevi	India	Karnataka	Female	1981-06-18
0018-NYROU	2020-10-09	Organic	2025-10-09	Standard	Annual	NaT	None	13.99	980	19	0	chitra	None	Telangana	Female	2004-12-01
0019-EFAEP	2022-08-25	Refferal	2024-08-25	Basic	Monthly	2024-10-31	Switched to competitor	12.99	160	76	1	raju	India	Meghalaya	Female	1992-04-25
0019-GFNTW	2019-11-11	Paid	2025-11-11	Premium	Annual	NaT	None	92.99	2185	3	0	Madhav	India	Uttar Pradesh	Male	1979-11-11
0020-INWCK	2023-05-06	Organic	2025-05-06	Standard	Monthly	NaT	None	13.99	640	58	0	parvati	India	Delhi	Female	1986-02-28
0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	Annual	NaT	None	20.99	550	62	0	rikim	India	Meghalaya	Female	1994-08-19
0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	Annual	NaT	None	13.99	840	27	0	vishakha	India	Rajasthan	Female	2000-09-02
0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	Monthly	2024-09-14	Forgot to cancel trial	16.99	42	99	1	raghvendra	India	Telangana	Male	1983-12-30
0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	Annual	NaT	None	22.99	1955	7	0	rishabh	India	Uttar Pradesh	Male	1991-05-14
0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	Monthly	NaT	None	13.99	790	47	0	sudevi	India	Maharashtra	Female	1977-10-06

In [100…

```
df_db_customer.head()
```

Out[100…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05

In [101…
```
df_db_subscription.head()
```

Out[101…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	None	13.99	627	12	0
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	None	6.99	210	34	0
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	None	22.99	1725	8	0
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

In [125…
```
#mutiple joins practice in pandas
#the best practice is to get this saved in a dataframe.
#first fix the support table duplicate columns and then merge it.
df = (df_db_subscription
                        .merge(df_db_customer, on = 'customerid', how = 'left')
                        .merge(df_db_support, on = 'customerid', how = 'left'))
```

In [103…
```
#VERY VERY IMPORTANT:
#This practice is done after we have joined the tables and now we will check the shape of the tables and joins created.
#We will check the shape and check if there is no duplicate values.
```

In [105…
```
df_db_subscription.shape
```

Out[105… (21, 12)

In [126…
```
df.shape #21 because we have added a new column of complaint_count hence the count increased.
```

Out[126… (21, 21)

In [108…
```
#there is a mismatch in the numbers there is 21 records(rows) in the subscription table and 23 rows in the combined table.
df_db_subscription['customerid'].nunique()
```

Out[108… 21

In [109…
```
df_db_customer['customerid'].nunique()
```

Out[109… 21

In [111…
```
df_db_support['customerid'].nunique()
```

Out[111… 7

In [112…
```
df_db_support['customerid'].size
```

Out[112… 9

In [113…
```
#here is the problem there are 7 unique values in the table but the size is 9, hence there are 2 entries that are picked and p
```

In [120…
```
#we can take the last record of the customerid which was repeated again and can work for it.

df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'].transform('count')
```

In [123…
```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('customerid', keep = 'last')
#this will sort all the values and will keep the last value in it and remove the duplicate records.
```

In [124…
```
df_db_support['customerid'].size
```

Out[124… 7

In [127…
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 21 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     object
 1   subscription_start_date  21 non-null     datetime64[ns]
 2   subscription_type        21 non-null     object
 3   renewal_date             21 non-null     datetime64[ns]
 4   plan_type                21 non-null     object
 5   contract_type            21 non-null     object
 6   cancellation_date        6 non-null      datetime64[ns]
 7   cancellation_reason      6 non-null      object
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
 11  churn_flag               21 non-null     int64
 12  customer_name            21 non-null     object
 13  country                  18 non-null     object
 14  state                    21 non-null     object
 15  gender                   21 non-null     object
 16  dob                      21 non-null     datetime64[ns]
 17  complaint_date           7 non-null      datetime64[ns]
 18  escalations              7 non-null      object
 19  csat_score               7 non-null      float64
 20  complaint_count          7 non-null      float64
dtypes: datetime64[ns](5), float64(3), int64(3), object(10)
memory usage: 3.6+ KB
```

In [128…
```
#we will create an excel sheet for the churn rate value and for easy value exploration.
df.to_csv('exported_churn_data.csv', index = False)
```

In [129…
```
#DATA ANALYSIS:
df.columns
```

Out[129…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='object')
```

In [133…
```
#1.Churn Rate - The number who are leaving and cancelling the subscription.
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

```
Churn Rate =  28.57 %
```

In [135…
```
#2. Retention Rate - The number of user who are renewing there subscription again.
retention_rate = 100- churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

```
Retention Rate =  71.43 %
```

In [138…
```
#3. churn by plan type: which type of user are leaving our platform.
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).reset_index(name = 'churn_rate_pct'))
#this will give us a percentage that from which plan people are leaving the most, just by seeing there percentage.

print(churn_by_plan)
```

```
  plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [139…
```
#4.a : churn by state + sum(revenue) & count of users
#4.b : churn by subscription type + sum(revenue) & count of users
```

In [140…
```
#5. ARPU = avg revenue per unit
df.columns
```

Out[140…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='object')
```

In [143…
```
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu, 2))
```

```
ARPU =  18.85
```

In [151…
```
#6. Avg tenure
# count of days users has used our service : cancellation date else current date
today = pd.Timestamp.today()
```

```
df['tenure_days'] = np.where(
        df['cancellation_date'].notna(),
        (df['cancellation_date'] - df['subscription_start_date']).dt.days,
        (today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print('Avg Tenure (Days) = ', round(avg_tenure), 0)
```

```
Avg Tenure (Days) =  1492 0
```

In [148…
```
#customer age:
today = pd.Timestamp.today()

df_db_customer['age'] = ((today - df_db_customer['dob']).dt.days / 365.25).astype(int)
```

In [149…
```
df_db_customer.head(4)
```

Out[149…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>age</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> </tbody> <tbody> </tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12	44
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23	30
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15	48
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30	24

In [152…
```
#7. Revenue Loss due to churn loss or the people leaving or cancelling the subscription:
revenue_at_risk = df.loc[df['churn_flag'] == 1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

```
Revenue at Risk (Rs 'K') = 73.94
```

In [154…
```
#8. Escalation Rate: If our problem instead of being solved grows to another new level, we call it escalation.
df['escalations'].unique()
#means for escalation there are only 2 values T or F
```

Out[154…
```
array([nan, 'Y', 'N'], dtype=object)
```

In [156…
```
escalation_rate = (df['escalations'] == 'Y').mean() * 100
print("Escalation Rate = ", round(escalation_rate, 2), "%")
```

```
Escalation Rate =  19.05 %
```

In [157…
```
# WHAT ARE KPI'S IN DATA ANALYSIS:
#Key Performance Indicators (KPIs) in data analysis are quantifiable metrics used to track and evaluate how effectively a busi
```

In [158…
```
#9. Avg Complaint Rate:-> users who complainted / total customers
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Complaints Per User = ", round(avg_complaints, 2))
```

```
Avg Complaints Per User =  0.43
```

In [165…
```
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) #encoding string to int
```

In [170…
```
#10. Correlation Escalation vs Churn
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])

print("Correlation between escalations and churn is = ", round(correlation, 2))
```

```
Correlation between escalations and churn is =  0.77
```

In [173…
```
#11. Creating a new column using the existing column called Churn Risk.
conditions = [
            (df['churn_score'] < 50),
            (df['churn_score'] >= 50) & (df['churn_score'] < 70),
            (df['churn_score'] >= 70)
] #we are passing this in the list as we can see the square brackets.

choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default = 'unknown')
```

In [174…
```
df[['churn_risk', 'churn_score']].tail()
```

Out[174…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>churn_risk</th> <th>churn_score</th> </thead> <tbody> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

med	62
low	27
high	99
low	7
low	47

In [175…

```
#4. VISUALISATION USING MATPLOTLIB & SEABORN.
df.head(3)
```

Out[175…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>...</th> <th>state</th> <th>gender</th> <th>dob</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> <th>tenure_days</th> <th>escalation</th> <th>churn_risk</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	None	13.99	627	...	Maharashtra	Male	1982-04-12	NaT	0	NaN	NaN	19
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Karnataka	Male	1995-11-23	2024-08-28	1	10.0	2.0	15
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	None	6.99	210	...	Delhi	Female	1978-02-15	NaT	0	NaN	NaN	13

3 rows × 24 columns

In [176…

```
df_visual = df.copy()# i want to keep the df as it is.
```

In [178…

```
df_visual.copy()
```

Out[178…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>

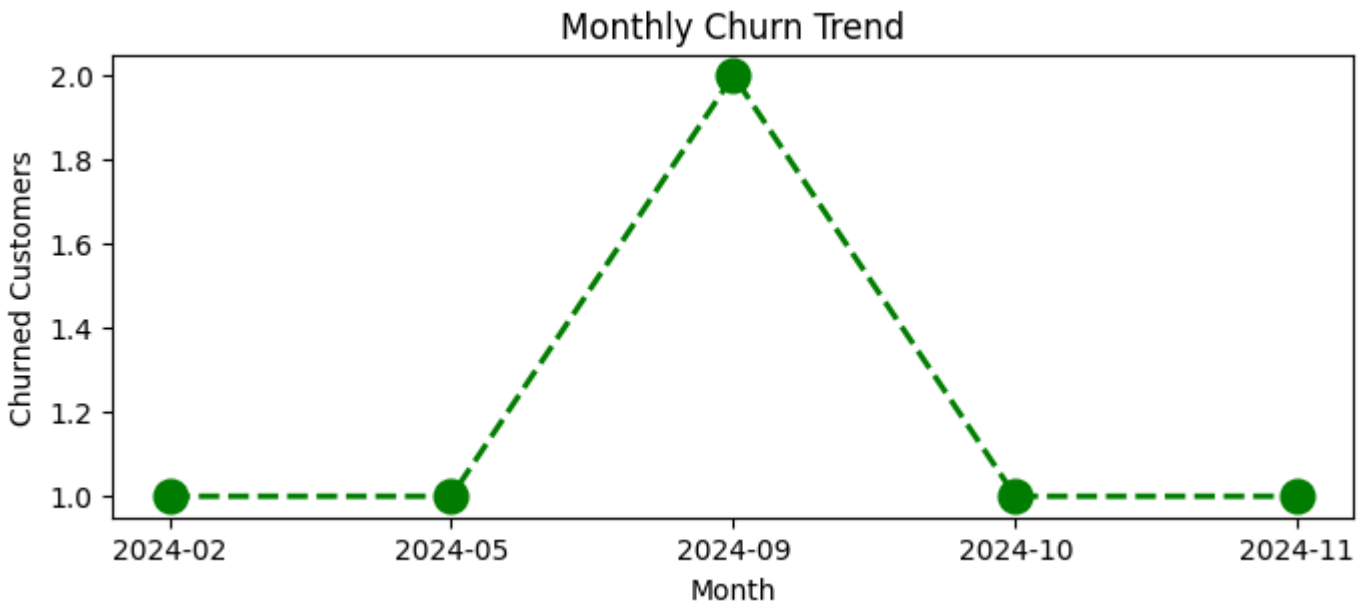
	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	...	state	gender	dob	complaint_date	escalations	csat_score	complaint_count	tenure_days	escalation	churn_risk
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	None	13.99	627	...	Maharashtra	Male	1982-04-12	NaT	0	NaN	NaN	1		
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	...	Karnataka	Male	1995-11-23	2024-08-28	1	10.0	2.0	1		
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	None	6.99	210	...	Delhi	Female	1978-02-15	NaT	0	NaN	NaN	1		
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	None	22.99	1725	...	Nagaland	Male	2001-08-30	NaT	0	NaN	NaN	2		
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	...	Delhi	Female	1990-05-05	2024-01-20	1	20.0	1.0			
5	0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	Annual	NaT	None	17.99	720	...	Delhi	Female	1988-12-10	2025-03-18	0	90.0	1.0	1		
6	0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	Monthly	2024-11-15	Not enough content	8.99	230	...	Meghalaya	Female	1976-09-21	2024-11-01	0	30.0	1.0	1		
7	0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	Annual	NaT	None	22.99	1840	...	Rajasthan	Male	1999-03-14	NaT	0	NaN	NaN	2		
8	0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	Monthly	NaT	None	13.99	240	...	Kathmandu	Female	1985-07-07	NaT	0	NaN	NaN	1		
9	0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	Annual	NaT	None	6.99	335	...	Kathmandu	Male	1993-10-29	NaT	0	NaN	NaN	1		
10	0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	Annual	NaT	None	22.99	1610	...	Maharashtra	Male	1997-01-22	NaT	0	NaN	NaN	1		
11	0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	Monthly	2024-05-01	Poor streaming quality	7.99	270	...	Karnataka	Female	1981-06-18	2024-04-10	1	25.0	1.0			
12	0018-NYROU	2020-10-09	Organic	2025-10-09	Standard	Annual	NaT	None	13.99	980	...	Telangana	Female	2004-12-01	NaT	0	NaN	NaN	2		
13	0019-EFAEP	2022-08-25	Refferal	2024-08-25	Basic	Monthly	2024-10-31	Switched to competitor	12.99	160	...	Meghalaya	Female	1992-04-25	2024-09-27	1	30.0	1.0			
14	0019-GFNTW	2019-11-11	Paid	2025-11-11	Premium	Annual	NaT	None	92.99	2185	...	Uttar Pradesh	Male	1979-11-11	NaT	0	NaN	NaN	2		
15	0020-INWCK	2023-05-06	Organic	2025-05-06	Standard	Monthly	NaT	None	13.99	640	...	Delhi	Female	1986-02-28	NaT	0	NaN	NaN	1		
16	0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	Annual	NaT	None	20.99	550	...	Meghalaya	Female	1994-08-19	NaT	0	NaN	NaN	1		
17	0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	Annual	NaT	None	13.99	840	...	Rajasthan	Female	2000-09-02	NaT	0	NaN	NaN	1		
18	0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	Monthly	2024-09-14	Forgot to cancel trial	16.99	42	...	Telangana	Male	1983-12-30	2024-09-14	0	90.0	2.0			
19	0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	Annual	NaT	None	22.99	1955	...	Uttar Pradesh	Male	1991-05-14	NaT	0	NaN	NaN	2		
20	0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	Monthly	NaT	None	13.99	790	...	Maharashtra	Female	1977-10-06	NaT	0	NaN	NaN	1		

21 rows × 24 columns

In [182…

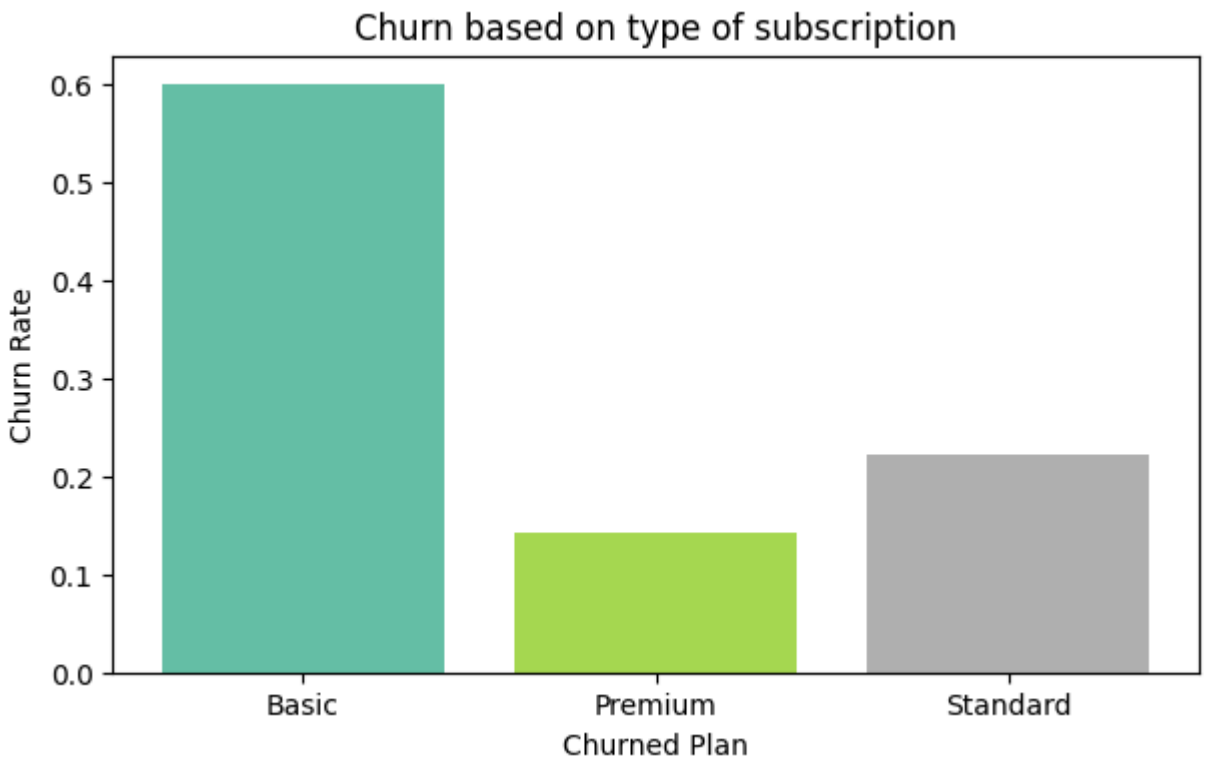
```
#4.1: Monthly churn Trend (Time Series KPI):
df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M') # M: Monthly frequency.

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month').size()

plt.figure(figsize = (8, 3))

plt.plot(churn_trend.index.astype(str), churn_trend.values, color = 'green', marker = 'o', linestyle = 'dashed', linewidth = 2

#Adding the title:
```

```
plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```



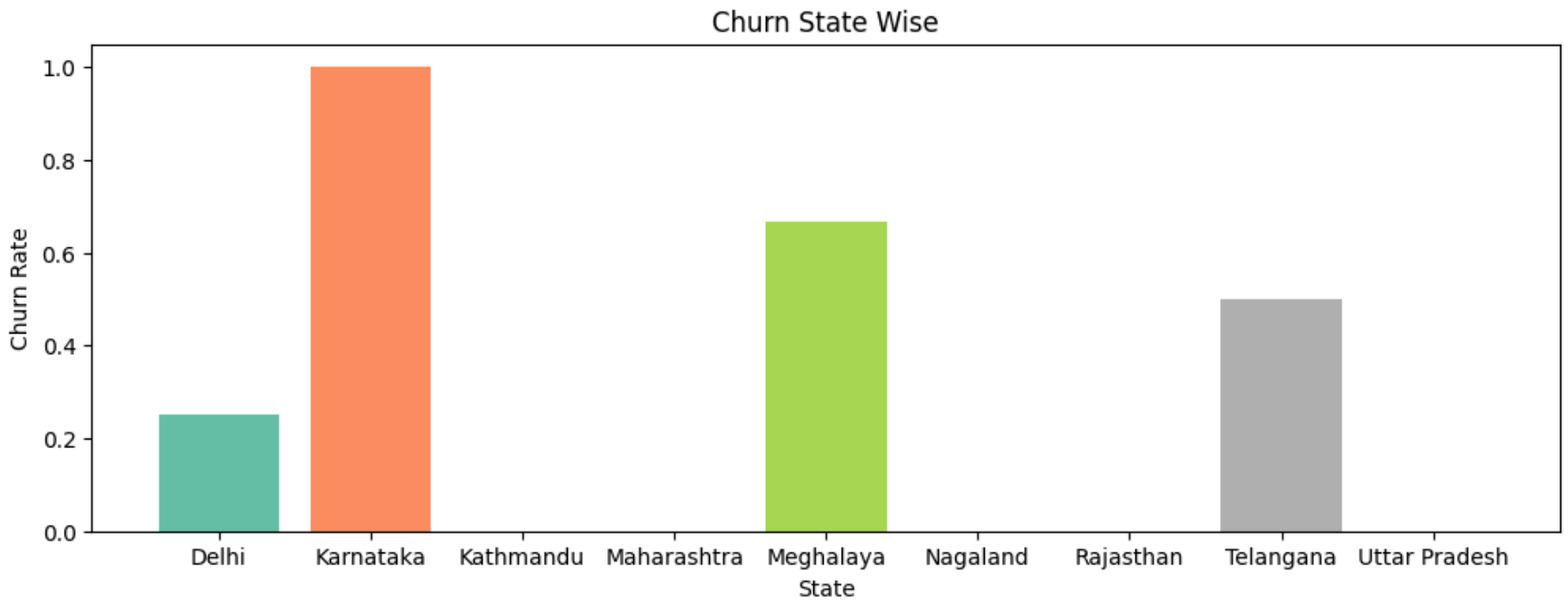
In [188…

```
#4.2: Churn By plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

#colors = ['yellow', 'purple', 'blue'] if there will be more than 3 values than this will be a trouble for us to be saved from

colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize = (7, 4))

plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn based on type of subscription')
plt.xlabel('Churned Plan')
plt.ylabel('Churn Rate')
plt.show()
```



In [190…

```
#4.3 Churn by states
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

#colors = ['yellow', 'purple', 'blue'] if there will be more than 3 values than this will be a trouble for us to be saved from

colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize = (12, 4)) #increased the lenth of the chart for better visualization.

plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn State Wise')
plt.xlabel('State')
plt.ylabel('Churn Rate')
plt.show()
```

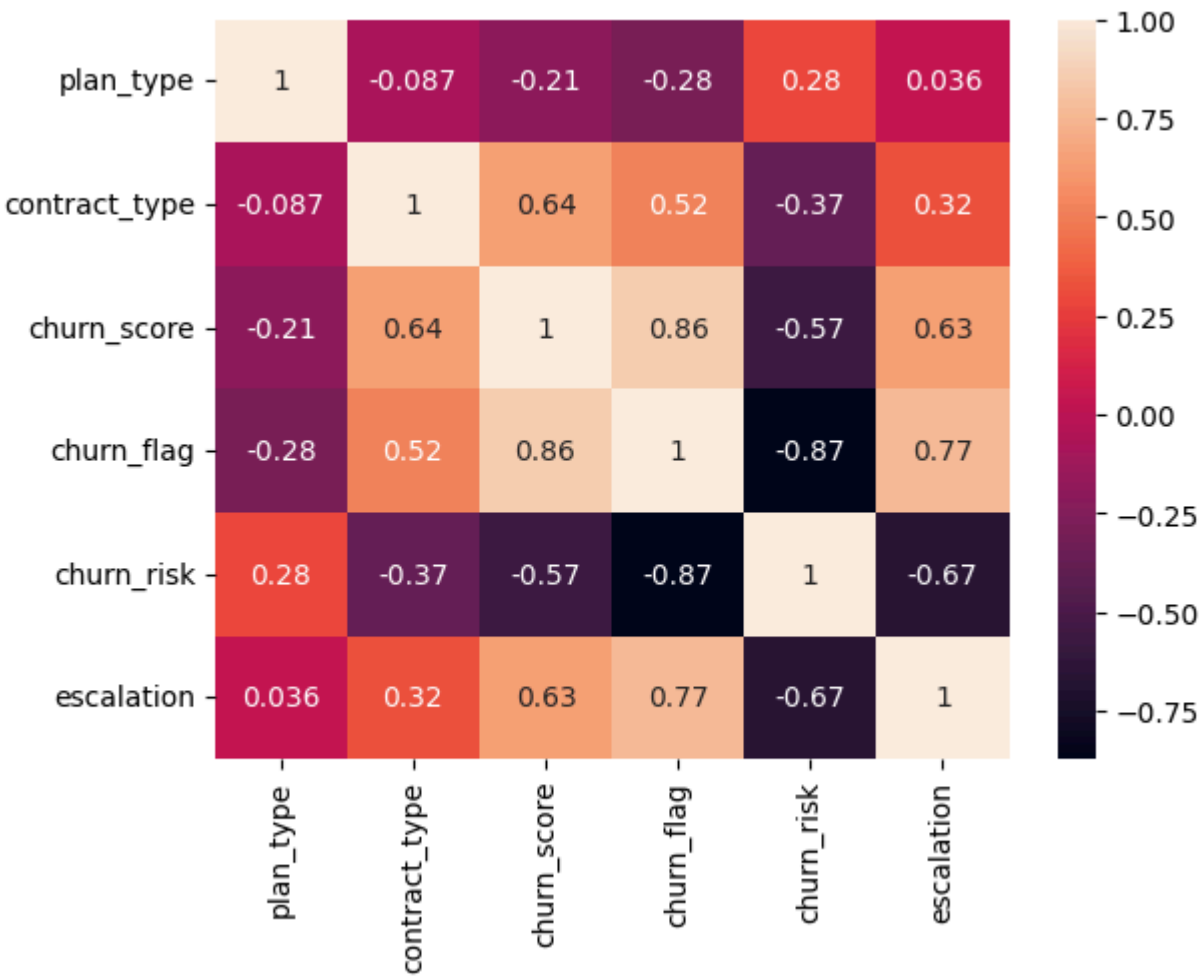


In [200…
```
# Visualization using Seaborn

#since heatmap can take only the numeric values hence for creation of a correlation we have to convert the values to integer.


#Encode: convert the string to numeric so that we can find corr between features.
#this is the wrong method of encoding: as numbers are not assigned based on priority.
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalation']]

categorical_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorical_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes #this will change the data to categorical codes.
```

In [197…
```
# To ignore such warnings we can use the following statement as:
import warnings
warnings.filterwarnings("ignore")
```

In [201…
```
 #Now creating the heatmap:
sns.heatmap(df_encoded.corr(), annot = True)
```

Out[201… <Axes: >



In [199…
```
df_encoded.head()
```

Out[199…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>contract_type</th> <th>churn_score</th> <th>churn_flag</th> <th>churn_risk</th> <th>escalation</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

2	0	12	0	1	0	
1	0	91	1	0	1	
0	1	34	0	1	0	
1	0	8	0	1	0	
2	1	88	1	0	1	

In [212…

```
#Correct Method of encoding - based on priority:
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_risk', 'escalation']]

order_mappings = {
                    'plan_type' : ['Basic', 'Standard', 'Premium'],
                    'contract_type' : ['Monthly', 'Annual'],
                    'churn_risk' : ['low', 'medium', 'high']
                    }

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories = order, ordered = True).codes #this will
```

In [203…

```
df_visual['plan_type'].unique()
```

Out[203…

```
array(['Standard', 'Premium', 'Basic'], dtype=object)
```

In [205…

```
df_visual['contract_type'].unique()
```

Out[205…

```
array(['Annual', 'Monthly'], dtype=object)
```

In [206…

```
df_visual['churn_risk'].unique()
```

Out[206…

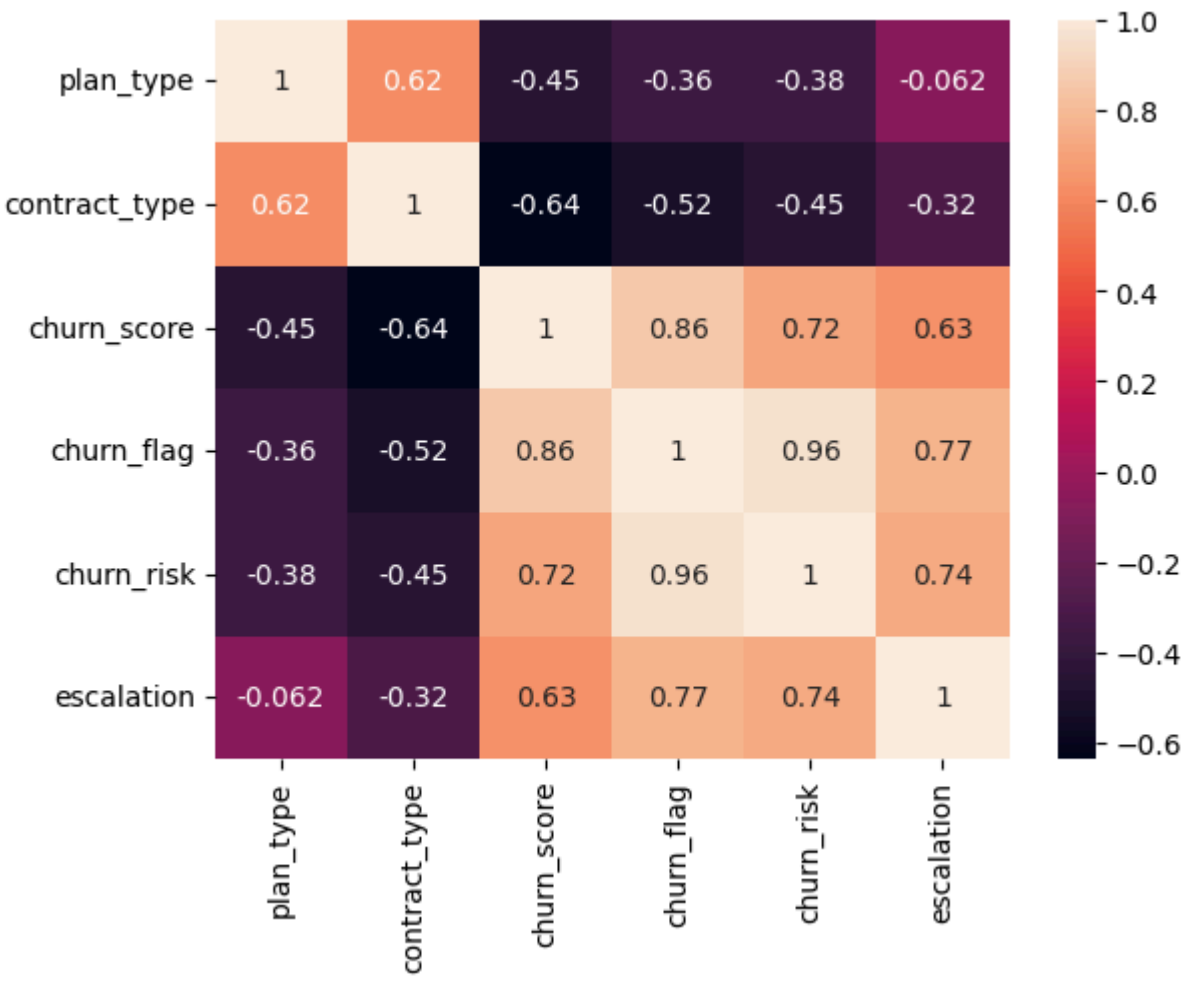
```
array(['low', 'high', 'med'], dtype=object)
```

In [215…

```
#heatmap(correlation matrix):
sns.heatmap(df_encoded.corr(), annot = True)
#the making of heatmap is very difficult in matplotlib, here in seaborn we just used the correlation between the columns and f
#DISTRIBUTION AND RESULATION CHARTS ARE VERY EASY IN SEABORN.
```

Out[215…

```
<Axes: >
```



In [216…

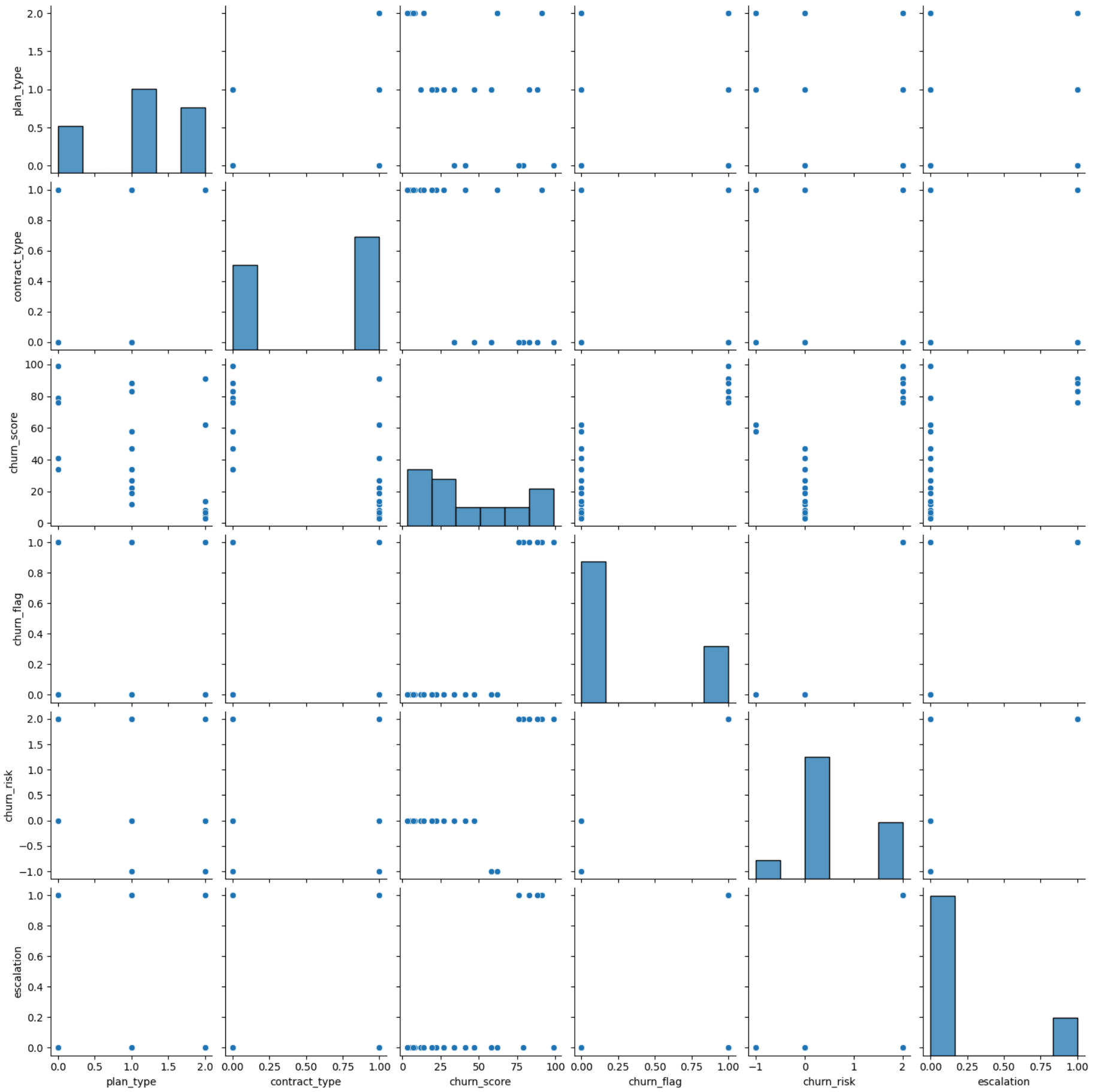
```
#We can do the further visualisation using Matplotlib as well but it makes our life much more difficult.
```

In [217…

```
#Pairplot - used in realtionship between the data.
sns.pairplot(df_encoded)
```

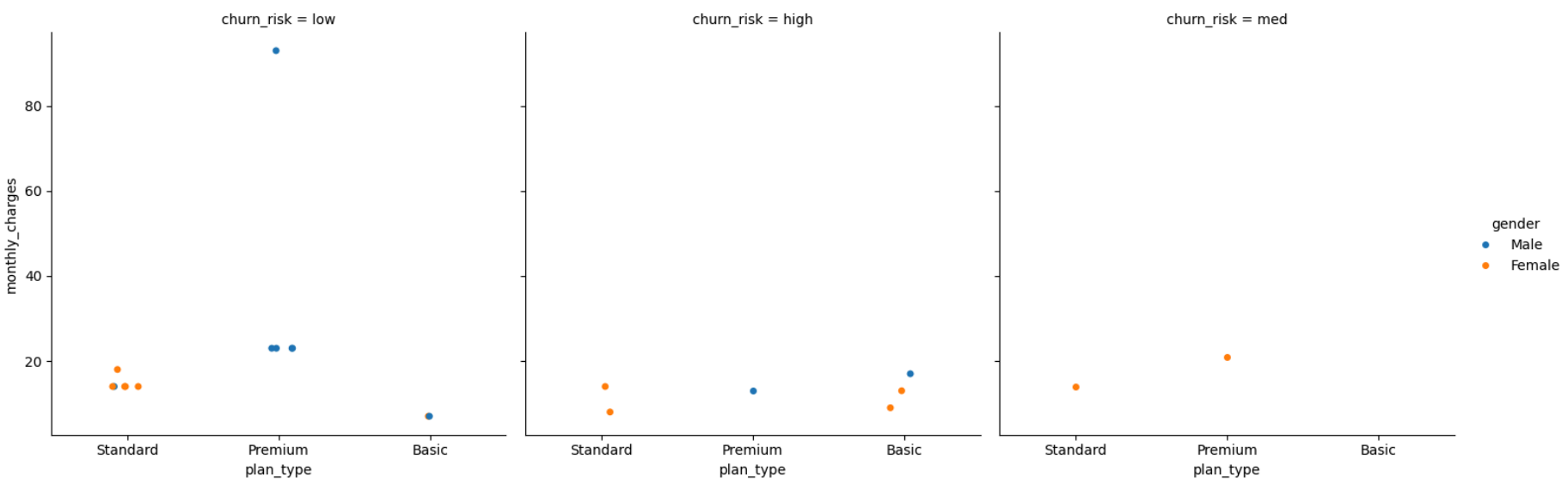
Out[217…

```
<seaborn.axisgrid.PairGrid at 0x2494d3ee560>
```



In [220…
```
#catplot / Facegrid plot - CATRGORICAL PLOT used to plot different categories in one plot.
#This is multidimension Comparison.
sns.catplot(data = df_visual,
            x = 'plan_type',
            y = 'monthly_charges',
            hue = 'gender',
            col = 'churn_risk')
```

Out[220…
```
<seaborn.axisgrid.FacetGrid at 0x2494d715ae0>
```



In [219…
```
df_visual.columns
```

Out[219…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'escalation', 'churn_risk', 'cancellation_month'],
      dtype='object')
```

In [223…
```
#PIVOT TABLE:
pd.pivot_table(
    df_visual,
    values='churn_flag',
    index='plan_type',
    aggfunc='mean'
).reset_index()
```

Out[223…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

Basic	0.600000	
Premium	0.142857	
Standard	0.222222	

In [225…
```
pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'churn_flag'],
    aggfunc={
            'monthly_charges' : 'sum',
            'customerid' : 'nunique', #here i made a mistake of writing nunique to nunqiue
            'churn_flag' : 'mean'
    }
)
```

Out[225…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th> <th>plan_type</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Basic</th> <th>Premium</th> <th>Standard</th> </tbody> <tbody></tbody>

0.600000	5	52.95					
0.142857	7	218.93					
0.222222	9	123.91					

In [226…
```
#PYTHON WITH SQL:
#working with python along with Python(Pandas)
```

In [236…
```
#create a database in SQL:
conn = sqlite3.connect('test_database.sqlite')

#table details for database:
conn.execute('CREATE TABLE users (first_name TEXT, country TEXT, budget INTEGER)')

#commit and save
conn.commit()
```

In [228…
```
#Path for the churn analysis file and the database created.
import os

print(os.getcwd())
```

```
C:\Users\hp\churn_analysis
```

In [229…
```
import os

print(os.path.abspath('customers.csv'))
```

```
C:\Users\hp\churn_analysis\customers.csv
```

In [230…
```
import os

print(os.path.abspath('test_database.sqlite'))
```

```
C:\Users\hp\churn_analysis\test_database.sqlite
```

In [237…
```
#insert data inside the database

cursor = conn.cursor()

#write a sql query to insert data in tables.
cursor.execute(
        """
            INSERT INTO users VALUES
```

```
                ('Madhav' , 'India', 5000),
                ('Rishabh', 'Germany', 2500),
                ('Shaurya', 'USA', 9000)
        """
)

#Commit and save:
conn.commit()

print("Data Inserted Successfully!")
```

```
Data Inserted Successfully!
```

In [238…

```
#check inserted data in the database
conn = sqlite3.connect('test_database.sqlite')
query = """SELECT * FROM users"""

df_results = pd.read_sql(query, conn)

df_results.head()
```

Out[238…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>first_name</th> <th>country</th> <th>budget</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

0	Madhav	India	5000
1	Rishabh	Germany	2500
2	Shaurya	USA	9000

In [239…

```
#Aggregation:
query = """
        SELECT country, SUM(budget) as total_budget
        FROM users
        GROUP BY country
"""

df_agg = pd.read_sql(query, conn)
df_agg
```

Out[239…

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>country</th> <th>total_budget</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

	country	total_budget
0	Germany	2500
1	India	5000
2	USA	9000

In [ ]:

```
#Always close the connection whenever the work is over.
conn.close()
```